

Estructura de datos

Colas

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

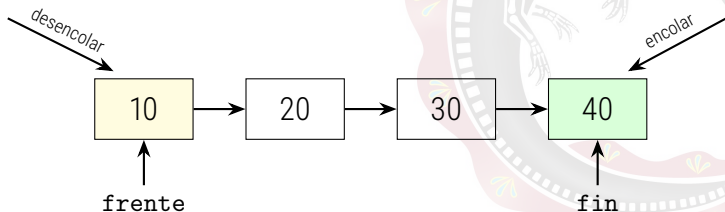
- 1 TDA cola
- 2 Implementación Cola enlazada
- 3 Encolar
- 4 Desencolar
- 5 Frente
- 6 Cola vacía
- 7 Limpiar



Tipo de dato abstracto Cola

Definición

Una **cola** es una **estructura de datos lineal** en la que los elementos se **insertan por un extremo**, denominado **fin**, y se **eliminan por el extremo opuesto**, denominado **frente**.



Principio FIFO (First In, First Out)

El **primer elemento** en **entrar** es el **primero en salir**.

Operaciones del TDA Cola

Las **operaciones definidas** para el TDA son:

encolar(elemento) **Agrega** un elemento **al final** de la **cola**.

desencolar() **Elimina y devuelve** el elemento ubicado **al frente**.

frente() **Consulta** el elemento del **frente** **sin eliminarlo**.

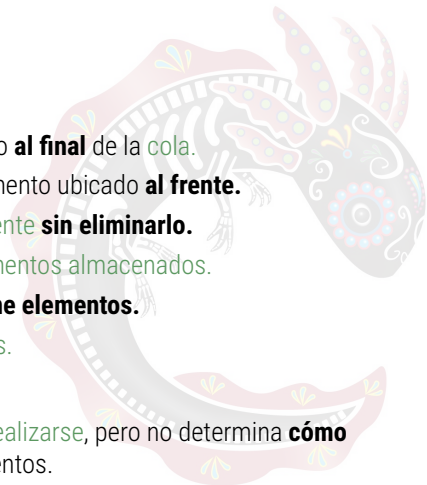
tamano() **Devuelve** el número de **elementos almacenados**.

estaVacia() Determina si la cola **contiene elementos**.

limpiar() **Elimina todos** los **elementos**.

Abstracción

El TDA establece **qué operaciones pueden realizarse**, pero no determina **cómo deben almacenarse** internamente los elementos.



Interfaz Cola<E>

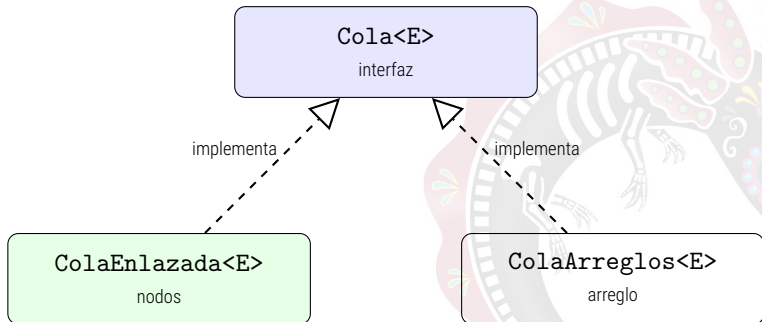
```
public interface Cola<E> {  
  
    void encolar(E elemento);  
    E desencolar();  
    E frente();  
    int tamano();  
    boolean estaVacía();  
    void limpiar();  
}
```

Contrato

La **interfaz** define el **comportamiento observable** de una cola **independientemente** de la **representación utilizada** para implementarla.



Interfaz e implementación

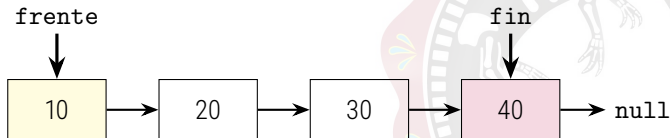


¡Importante!

El comportamiento FIFO **no depende** de que la cola se **implemente** mediante **nodos** o mediante **un arreglo**.

Cola enlazada

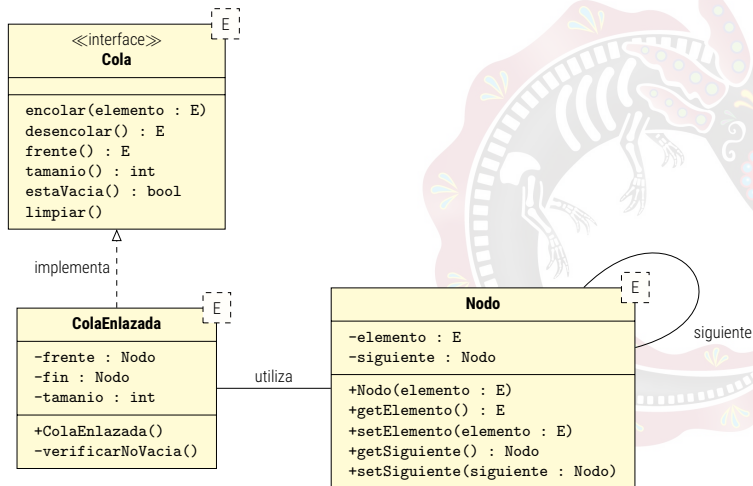
`ColaEnlazada<E>` utiliza objetos `Nodo<E>` conectados mediante la referencia `siguiente`.



Decisión de diseño

Se conservan referencias al **primer** y al **último nodo** para evitar recorridos innecesarios.

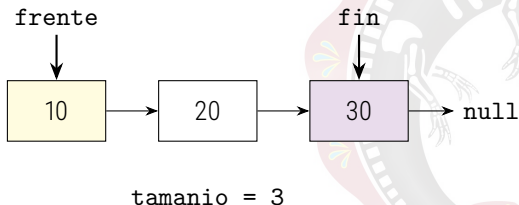
Diagrama de clases



Representación interna

`ColaEnlazada<E>` mantiene **tres atributos**:

- **frente**: referencia al **primer nodo**.
- **fin**: referencia al **último nodo**.
- **tamaño**: número de **elementos almacenados**.



Invariantes

- Si la cola no está vacía:

`frente != null,      fin != null,      fin.siguiente == null`

Constructor

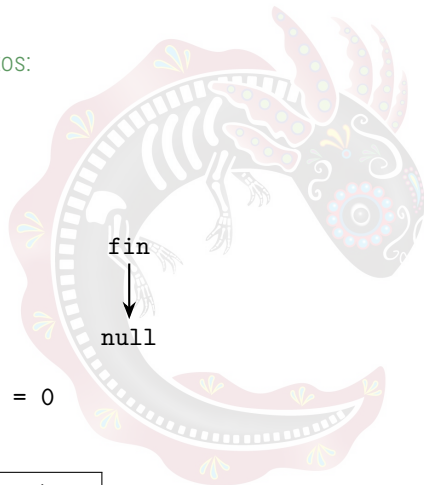
Una cola **recién creada** no contiene elementos:

- 1: $frente \leftarrow null$
- 2: $fin \leftarrow null$
- 3: $tamaño \leftarrow 0$

frente
↓
null

tamaño = 0

estaVacia() = true



Método `encolar(elemento)`

Require: elemento

- 1: $nuevo \leftarrow \text{Nodo}(\text{elemento})$
- 2: **if** la cola está vacía **then**
- 3: $frente \leftarrow nuevo$
- 4: $fin \leftarrow nuevo$
- 5: **else**
- 6: $fin.siguiente \leftarrow nuevo$
- 7: $fin \leftarrow nuevo$
- 8: **end if**
- 9: $tamano \leftarrow tamano + 1$



¡Importante!

La referencia `fin` permite **insertar sin recorrer** la estructura:

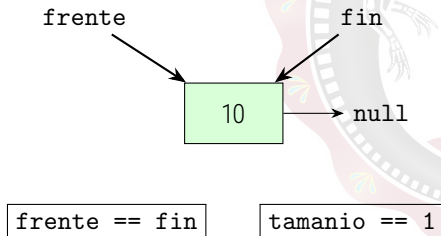
$$\text{encolar}() = O(1)$$

Primer elemento de la cola

Al ejecutar:

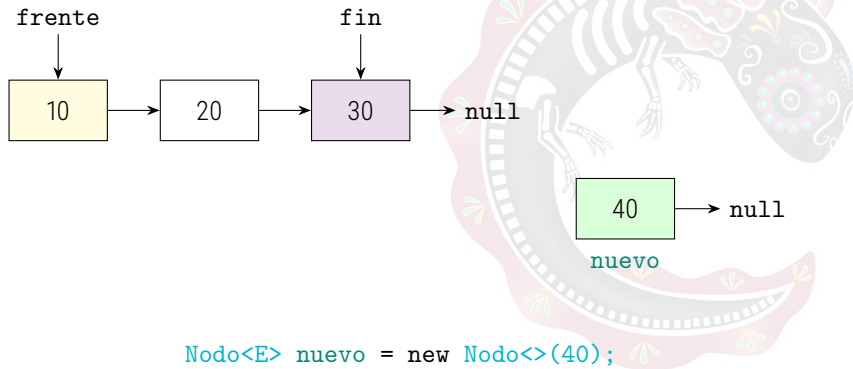
`encolar(10)`

el nuevo nodo **debe convertirse simultáneamente** en **frente** y **fin**.



encolar(40): crear el nodo

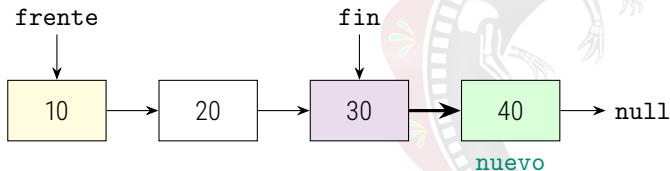
Estado inicial:



encolar(40): crear el enlace

Se ejecuta:

```
fin.setSiguiente(nuevo)
```

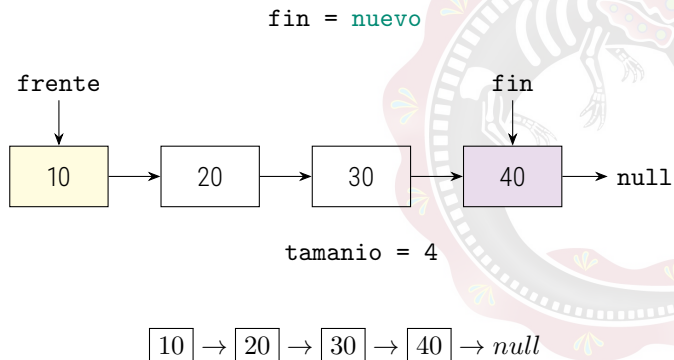


Estado intermedio

El nuevo nodo **ya forma parte** de la estructura, pero **fin todavía** referencia al nodo 30.

encolar(40): actualizar fin

Finalmente:



¿Por qué conservar la referencia `fin`?

Consideremos la operación `encolar()`:

Solo *frente*

Para localizar el **último nodo**:

frente $\rightarrow \dots \rightarrow$ *ultimo*

Es necesario **recorrer la cola**:

$$O(n)$$

frente **y** *fin*

Accedemos directamente al último nodo:

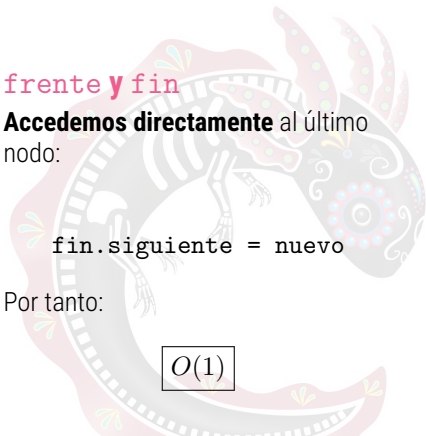
`fin.siguiente = nuevo`

Por tanto:

$$O(1)$$

¡Importante!

Una **referencia adicional** permite *evitar* el **recorrido completo** de la estructura durante la inserción.



Método desencolar()

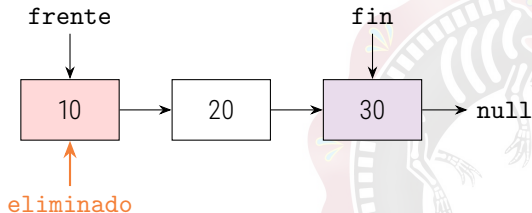
```
1: verificar que la cola no esté vacía
2:  $\text{eliminado} \leftarrow \text{frente}$ 
3:  $\text{frente} \leftarrow \text{frente.siguiente}$ 
4:  $\text{tamanio} \leftarrow \text{tamanio} - 1$ 
5: if  $\text{tamanio} = 0$  then
6:    $\text{fin} \leftarrow \text{null}$ 
7: end if
8: return  $\text{eliminado.elemento}$ 
```

$$\text{desencolar}() = O(1)$$



desencolar(): conservar el frente

Partimos de:



`eliminado = frente`

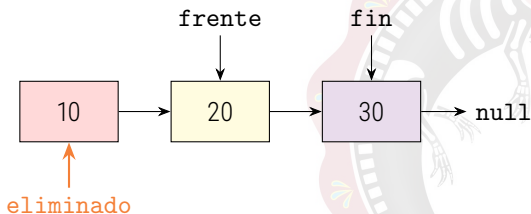
¿Por qué conservarlo?

Necesitamos mantener una **referencia** al nodo para poder devolver posteriormente su **elemento**.

desencolar(): actualizar el frente

Se ejecuta:

```
frente = frente.getSiguiente()
```



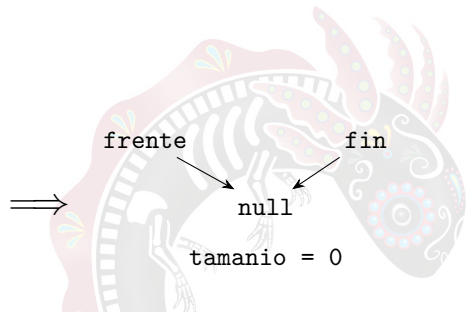
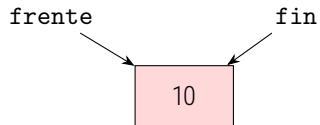
`desencolar() = 10`

La cola queda:

`20 → 30 → null`

Eliminar el único elemento

Consideremos:



¡Importante!

Al eliminar el **último elemento**, actualizar **frente** no es suficiente. También debemos establecer:

```
fin = null
```

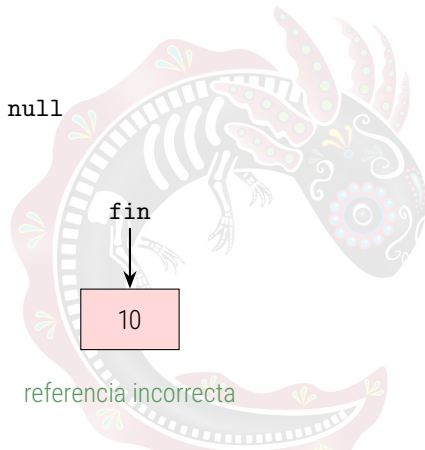
¿Qué ocurriría si no actualizamos `fin`?

Después de **eliminar** el único nodo:

```
frente = null
```

pero, si olvidamos modificar `fin`:

```
frente  
↓  
null
```



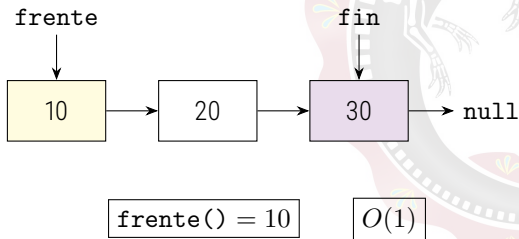
Estado inconsistente

Tendríamos una cola con `tamaño == 0`, pero con `fin != null`.

Método `frente()`

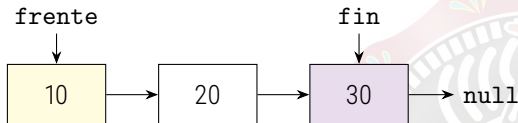
La operación **consulta** el **primer elemento** **sin modificar** la estructura.

- 1: verificar que la cola no esté vacía
- 2: **return** `frente.elemento`



frente() vs. desencolar()

Partimos de:



frente()

frente() = 10

La cola permanece:

10 → 20 → 30 → null

desencolar()

desencolar() = 10

La cola cambia:

20 → 30 → null

¡Importante!

Ambas operaciones **acceden al primer elemento**, pero únicamente **desencolar() modifica la cola**.

Cola vacía

Las operaciones:

`desencolar()` y `frente()`

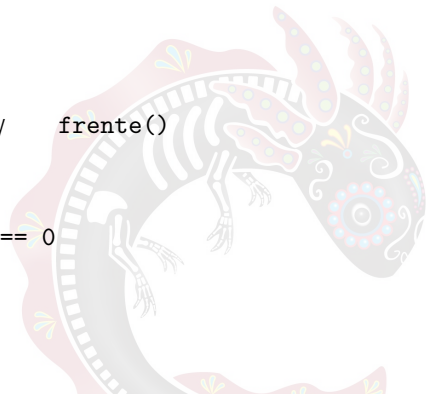
no son válidas cuando:

`tamano == 0`

- 1: **if** la cola está vacía **then**
- 2: lanzar `IllegalStateException`
- 3: **end if**

¡Importante!

`desencolar()` y `frente()` requieren que exista **al menos un elemento** en la cola.



Método `verificarNoVacia()`

La validación puede **encapsularse** en un método auxiliar:

- 1: **if** *tamano* = 0 **then**
- 2: lanzar `IllegalStateException`
- 3: **end if**

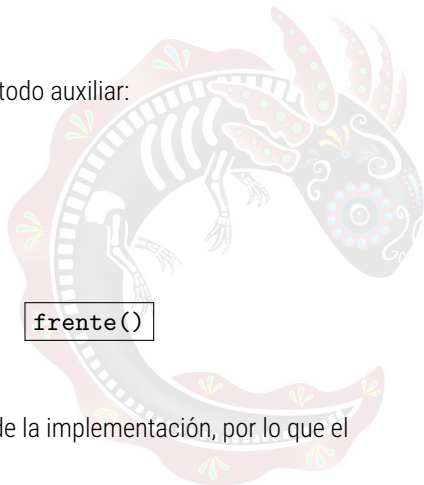
Este método es utilizado por:

`desencolar()`

`frente()`

Encapsulamiento

La validación constituye un **detalle interno** de la implementación, por lo que el método puede declararse como `private`.



Métodos `tamano()` y `estaVacia()`

`tamano()`

return *tamano*

10 → 20 → 30 → *null*

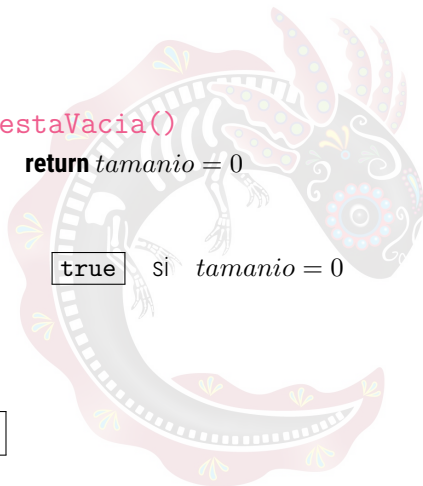
`tamano() = 3`

$O(1)$

`estaVacia()`

return *tamano* = 0

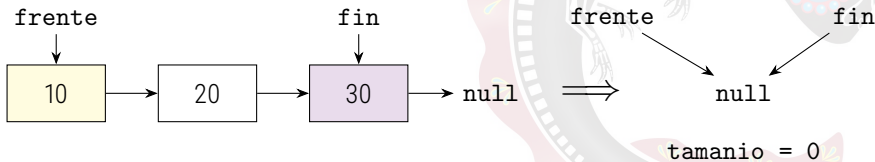
`true` si *tamano* = 0



Método limpiar()

Para eliminar todos los elementos:

- 1: $frente \leftarrow null$
- 2: $fin \leftarrow null$
- 3: $tamaño \leftarrow 0$



Administración de memoria

Los nodos **dejan de ser accesibles** desde la cola y **pueden ser recuperados** posteriormente por el **recolector de basura**.

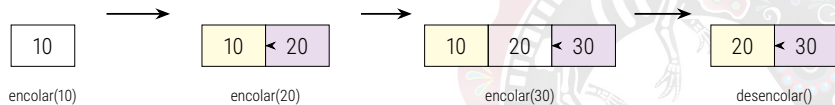
Ejemplo de operaciones

Operación	Cola	Resultado
encolar(10)	[10]	—
encolar(20)	[10, 20]	—
encolar(30)	[10, 20, 30]	—
frente()	[10, 20, 30]	10
desencolar()	[20, 30]	10
desencolar()	[30]	20

entrada: 10, 20, 30

salida: 10, 20, 30

Secuencia de operaciones



¡Importante!

Los elementos se **insertan** por el **fin** y se **eliminan** por el **frente**.

Invariantes de ColaEnlazada<E>

Después de cualquier **operación válida**:

- 1 Si la cola **está vacía**:

$$\text{tamano} == 0 \iff \text{frente} == \text{null} \wedge \text{fin} == \text{null}$$

- 2 Si existe un **único elemento**:

$$\text{frente} == \text{fin}$$

- 3 Si existen **elementos**:

$$\text{frente} != \text{null} \quad \text{fin} != \text{null}$$

- 4 El **último nodo** siempre **cumple**:

$$\text{fin.getSiguiete()} == \text{null}$$

Verificación

Estas propiedades **permiten comprobar** que `encolar()` y `desencolar()` conservan un **estado válido** de la cola.

Complejidad temporal

Operación	Complejidad
encolar(elemento)	$O(1)$
desencolar()	$O(1)$
frente()	$O(1)$
tamano()	$O(1)$
estaVacía()	$O(1)$
limpiar()	$O(1)$

¡Importante!

Las operaciones fundamentales **no requieren recorrer** la estructura enlazada.

Pila enlazada vs. cola enlazada

Característica	PilaEnlazada<E>	ColaEnlazada<E>
Componente básico	Nodo<E>	Nodo<E>
Referencias externas	cima	frente, fin
Inserción	Cima	Fin
Eliminación	Cima	Frente
Principio	LIFO	FIFO
Inserción	$O(1)$	$O(1)$
Eliminación	$O(1)$	$O(1)$

Misma estructura, distinto comportamiento

Pila

cima

Inserción y eliminación:

mismo extremo

LIFO

Cola

frente, fin

Inserción y eliminación:

extremos opuestos

FIFO

¡Importante!

Ambas implementaciones utilizan `Nodo<E>`; lo que cambia es **la organización de las referencias** y las **operaciones permitidas** por cada TDA.



¿Por qué Cola no extiende Lista?

Una **cola** **debe exponer**:

`encolar` `desencolar` `frente`

Una **lista** **permite** además:

`agregar(indice)` `obtener(indice)` `eliminar(indice)`

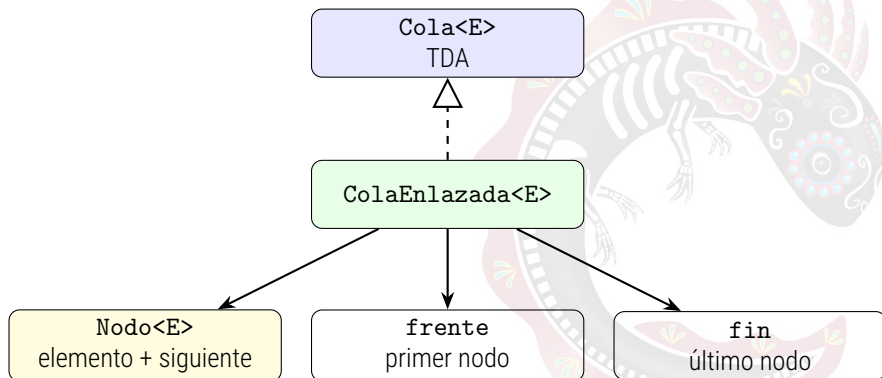
¡Importante!

Permitir modificaciones en **posiciones arbitrarias** **rompería** la **restricción FIFO** que caracteriza al TDA Cola.

Diseño

Podemos **reutilizar nodos y estructuras** enlazadas **sin exponer** operaciones que **no pertenecen** al TDA.

Implementación de ColaEnlazada<E>



Idea fundamental

ColaEnlazada<E> implementa el comportamiento **FIFO** manteniendo referencias directas al **primer** y al **último nodo**.

Fin de la presentación

¿Preguntas o comentarios?

